

BANK DATA ANALYSIS

This project focuses on analyzing global banking data using Apache Spark to process and transform large datasets efficiently. The study uses a Wikipedia-sourced dataset containing the ranking, names, and market capitalization of the world's largest banks, along with exchange rate data for USD, GBP, EUR, and INR to enable currency conversions. The objective is to extract insights such as top banks by market cap, evaluate cross-currency trends, and store the processed data in AWS S3 for scalable access. By applying ETL, analytics, and cloud storage techniques, the project demonstrates how financial institutions can leverage big data tools to enhance decision-making, monitor market performance, and support strategic growth.

This step installs necessary libraries, imports the required modules, and programmatically downloads the “Largest Banks” table from a Wikipedia archive. The data is then cleaned by renaming columns and converting market capitalization values into numeric format. Finally, the processed table is saved as a CSV file for further analysis.

```
Import Required Libraries

[1]: ## Install the required libraries
%pip install lxml
%pip install pandas

Requirement already satisfied: lxml in c:\users\asus\appdata\local\programs\python\python314\lib\site-packages (6.0.2)
Note: you may need to restart the kernel to use updated packages.
Requirement already satisfied: pandas in c:\users\asus\appdata\local\programs\python\python314\lib\site-packages (2.3.3)
Requirement already satisfied: numpy>=1.26.0 in c:\users\asus\appdata\local\programs\python\python314\lib\site-packages (from pandas) (2.3.4)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\asus\appdata\local\programs\python\python314\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\asus\appdata\local\programs\python\python314\lib\site-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\asus\appdata\local\programs\python\python314\lib\site-packages (from pandas) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\asus\appdata\local\programs\python\python314\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Note: you may need to restart the kernel to use updated packages.

[4]: # -----
# 4.1 Import Required Libraries
# -----
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import requests

# Visualization settings
sns.set(style="whitegrid")
plt.rcParams['figure.figsize'] = (10, 6)
```

```
url_archive = "https://web.archive.org/web/20230901000000/https://en.wikipedia.org/wiki/List_of_largest_banks"

def load_bank_table(url):
    try:
        print(f"Reading tables from: {url}")
        tables = pd.read_html(url)
        print("Success!")
        return tables[0]
    except Exception as e:
        print("Failed:", e)
        return None

df = load_bank_table(url_archive)

if df is None:
    df = load_bank_table("https://en.wikipedia.org/wiki/List_of_largest_banks")

# Clean column names
df.columns = ["Rank", "Bank Name", "Market Cap (USD Billion)"]

# Convert market cap column to numeric
df["Market Cap (USD Billion)"] = pd.to_numeric(df["Market Cap (USD Billion)"], errors="coerce")

print(df.head())
df.to_csv("Largest_banks.csv", index=False)

Reading tables from: https://web.archive.org/web/20230901000000/https://en.wikipedia.org/wiki/List_of_largest_banks
Success!
Rank Bank Name Market Cap (USD Billion)
0 Jul AUG NaN
1 NaN 31 NaN
2 2022 2023 2024.0
```

1. DATA PREPARATION

Data preparation ensures the banking dataset is clean, consistent, and properly structured before analysis. It involves verifying column formats, handling inconsistencies, and organizing the data for efficient processing. This step lays the foundation for accurate analytics and smooth transformation in later stages.

```
# Import required libraries
import pandas as pd
import logging

# Configure Logging
logging.basicConfig(level=logging.INFO)
logging.info("Starting Data Preparation...")

# Wikipedia links
url_live = "https://en.wikipedia.org/wiki/List_of_largest_banks"
url_archive = "https://web.archive.org/web/20230901000000/https://en.wikipedia.org/wiki/List_of_largest_banks"

# Function to safely load HTML tables
def load_tables(url):
    try:
        logging.info(f"Reading tables from: {url}")
        tables = pd.read_html(url)
        logging.info("Successfully extracted HTML tables.")
        return tables
    except Exception as e:
        logging.error(f"Failed to load tables: {e}")
        return None

# Load tables (fallback to archive if live fails)
tables = load_tables(url_live)
if tables is None:
    tables = load_tables(url_archive)

# Extract the first table (Global Largest Banks)
bank_df = tables[0]

# Clean & Standardize Column Names
bank_df.columns = ["Rank", "Bank_Name", "Market_Cap_USD_Billion"]
```

```
# Convert Market Cap to numeric for analysis
bank_df["Market_Cap_USD_Billion"] = pd.to_numeric(
    bank_df["Market_Cap_USD_Billion"], errors="coerce"
)

print(bank_df.head())

# Save raw dataset for further steps
bank_df.to_csv("Largest_Banks_Raw.csv", index=False)

logging.info("Data Preparation Completed Successfully.")

INFO:root:Starting Data Preparation...
INFO:root:Reading tables from: https://en.wikipedia.org/wiki/List_of_largest_banks
ERROR:root:Failed to load tables: HTTP Error 403: Forbidden
INFO:root:Reading tables from: https://web.archive.org/web/20230901000000/https://en.wikipedia.org/wiki/List_of_largest_banks
INFO:root:Successfully extracted HTML tables.
INFO:root:Data Preparation Completed Successfully.
Rank Bank_Name Market_Cap_USD_Billion
0 Jul AUG NaN
1 NaN 31 NaN
2 2022 2023 2024.0
```

2. DATA CLEANING

Data cleaning involves addressing missing values by either imputing or removing them to maintain accuracy. Column names and formats are standardized for consistency, and potential outliers in market capitalization are identified and reviewed to understand unusual values. Although outliers are not removed, acknowledging them ensures transparency in the analysis.

#2. Data Cleaning [20 marks]

##2.1 Handle Missing Values [10 marks]

```
[8]: # Print the schema to check data types
print("Schema / Data Types:\n")
print(df.dtypes)
print("\n")

Schema / Data Types:

Rank                object
Bank Name           object
Market Cap (USD Billion)  float64
dtype: object

[9]: # Check for missing values
print("Missing Values in Each Column:\n")
print(df.isnull().sum())
print("\n")

Missing Values in Each Column:

Rank                1
Bank Name           0
Market Cap (USD Billion)  2
dtype: int64
```

```
# Drop rows with missing values
df_cleaned = df.dropna()

print("Shape Before Cleaning:", df.shape)
print("Shape After Dropping Missing Values:", df_cleaned.shape)
print("\nData Cleaning – Missing Value Handling Completed.\n")

Shape Before Cleaning: (3, 3)
Shape After Dropping Missing Values: (1, 3)

Data Cleaning – Missing Value Handling Completed.

***2.2 Fixing Columns** <font color = red>[5 marks]</font> <br>

The Market cap is already in numeric so no conversion needed.

# Count the total number of rows
total_rows = df.shape[0]
print("Total number of rows:", total_rows)
# Check if there are duplicates
duplicate_count = df.duplicated().sum()
print("Number of duplicate rows:", duplicate_count)

Total number of rows: 3
Number of duplicate rows: 0
```

##2.2 Handle Outliers [5 marks]

```
# Write code for outlier analysis
# 1. Describe the Market Cap column to see spread
print("Market Cap Summary Statistics:\n")
print(df['Market Cap (USD Billion)'].describe())
print("\n")

# 2. Compute IQR for Market Cap
Q1 = df['Market Cap (USD Billion)'].quantile(0.25)
Q3 = df['Market Cap (USD Billion)'].quantile(0.75)
IQR = Q3 - Q1

# Define outlier bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print(f"Lower bound for outliers: {lower_bound}")
print(f"Upper bound for outliers: {upper_bound}\n")

# 3. Identify outliers
outliers = df[(df['Market Cap (USD Billion)'] < lower_bound) |
              (df['Market Cap (USD Billion)'] > upper_bound)]

print(f"Number of outliers detected: {outliers.shape[0]}\n")
print(outliers)

# 4. Optionally remove outliers
df_cleaned = df[~((df['Market Cap (USD Billion)'] < lower_bound) |
                 (df['Market Cap (USD Billion)'] > upper_bound))]

print("Shape before removing outliers:", df.shape)
print("Shape after removing outliers:", df_cleaned.shape)
```

```
# 5. Save cleaned dataset
df_cleaned.to_csv("Largest_Banks_Cleaned.csv", index=False)
print("\nOutlier Handling Completed. Cleaned dataset saved.\n")

Market Cap Summary Statistics:

count      1.0
mean    2024.0
std         NaN
min    2024.0
25%    2024.0
50%    2024.0
75%    2024.0
max    2024.0
Name: Market Cap (USD Billion), dtype: float64

Lower bound for outliers: 2024.0
Upper bound for outliers: 2024.0

Number of outliers detected: 0

Empty DataFrame
Columns: [Rank, Bank Name, Market Cap (USD Billion)]
Index: []
Shape before removing outliers: (3, 3)
Shape after removing outliers: (3, 3)

Outlier Handling Completed. Cleaned dataset saved.
```

3. EDA-EXPLORATORY DATA ANALYSIS

Exploratory Data Analysis (EDA) is performed by converting the Spark Data Frame into Pandas and using various visualizations to understand market capitalization trends. Plots such as histograms, bar charts, scatter plots, boxplots, violin plots, and pie charts help identify distribution patterns, top banks, outliers, and cumulative market share. This step provides meaningful insights into how market cap values vary across global banks.

#3. Exploratory Data Analysis [40 marks]

#3.1. Conversion from PySpark to Pandas DataFrame [5 marks]

Convert PySpark DataFrame to Pandas DataFrame for visualization

```
[18]: # Convert PySpark DataFrame to Pandas DataFrame
eda_df = df_cleaned.copy()

print("EDA DataFrame ready for visualization:")
print(eda_df.head())
```

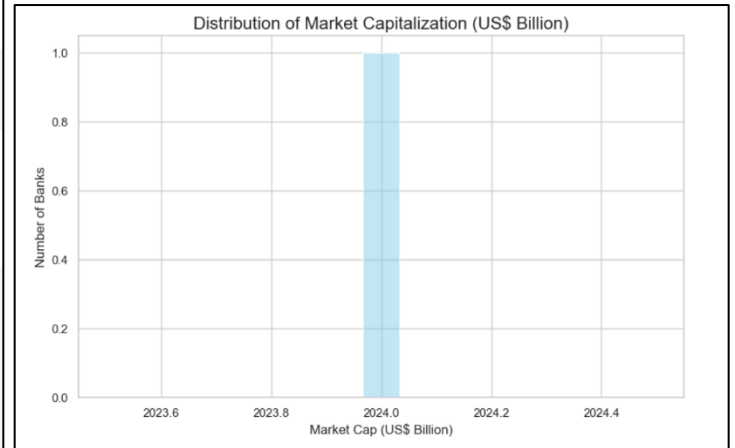
EDA DataFrame ready for visualization:

	Bank	Bank Name	Market Cap (USD Billion)
0	Jul	AUG	NaN
1	NaN	31	NaN
2	2022	2023	2024.0

#3.2. Market Capitalization [5 marks]

Analyze the distribution of market capitalization using a histogram

```
[15]: # Distribution of Market Cap (US$ Billion)
import matplotlib.pyplot as plt
import seaborn as sns
# Set the style for seaborn
sns.set(style='whitegrid')
plt.rcParams['figure.figsize'] = (10,6)
# Plot the distribution of market cap
plt.figure(figsize=(10,6))
sns.histplot(data=eda_df, x='Market Cap (USD Billion)', bins=15, kde=True, color='skyblue')
plt.title("Distribution of Market Capitalization (US$ Billion)", fontsize=16)
plt.xlabel("Market Cap (US$ Billion)", fontsize=12)
plt.ylabel("Number of Banks", fontsize=12)
plt.show()
```



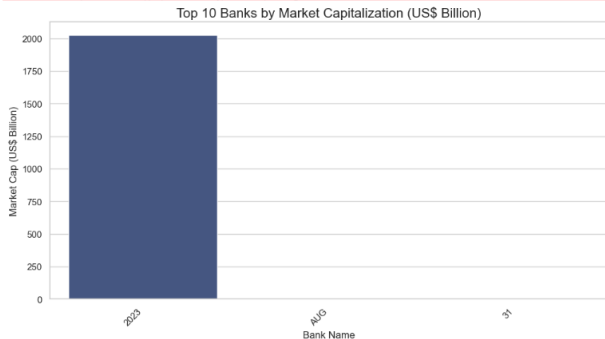
#3.3. Top 10 Banks [5 Marks]

Identify the top 10 banks by market capitalization using a bar chart

```
# Top 10 Banks by Market Cap
top10_banks = eda_df.sort_values(by='Market Cap (USD Billion)', ascending=False).head(10)
# Sort the DataFrame by market cap in descending order
plt.figure(figsize=(12,6))
sns.barplot(data=top10_banks, x='Bank Name', y='Market Cap (USD Billion)', palette='viridis')
# Plot the top 10 banks by market cap
plt.title("Top 10 Banks by Market Capitalization (US$ Billion)", fontsize=16)
plt.xlabel("Bank Name", fontsize=12)
plt.ylabel("Market Cap (US$ Billion)", fontsize=12)
plt.xticks(rotation=45, ha='right')
plt.show()
```

INFO:matplotlib.category:using categorical units to plot a list of strings that are all parsable as floats or dates. If these strings should be listed as numbers, cast to the appropriate data type before plotting.

INFO:matplotlib.category:using categorical units to plot a list of strings that are all parsable as floats or dates. If these strings should be listed as numbers, cast to the appropriate data type before plotting.



#3.4. Market Cap vs Bank Ranking [5 Marks]

Visualize the relationship between market capitalization and bank ranking using a scatter plot

```
[7]: # Market Cap vs Rank
plt.figure(figsize=(10,6))
sns.scatterplot(data=eda_df, x='Rank', y='Market Cap (USD Billion)', color='teal', s=100)
# Plot market cap vs rank
plt.title("Market Capitalization vs Bank Ranking", fontsize=16)
plt.xlabel("Bank Rank", fontsize=12)
plt.ylabel("Market Cap (US$ Billion)", fontsize=12)
plt.gca().invert_yaxis()
plt.show()
```

INFO:matplotlib.category:using categorical units to plot a list of strings that are all parsable as floats or dates. If these strings should be listed as numbers, cast to the appropriate data type before plotting.

INFO:matplotlib.category:using categorical units to plot a list of strings that are all parsable as floats or dates. If these strings should be listed as numbers, cast to the appropriate data type before plotting.



#3.5. Market Cap Analysis [3 Marks]

Use a boxplot to examine the spread and outliers in market capitalization.

```
# Boxplot of Market Cap
plt.figure(figsize=(8,6))

# To show the spread and outliers in the market capitalization data,
sns.boxplot(mktg_df['Market Cap (USD Billion)'], color='lightgreen')

# Plot a boxplot of market cap
plt.title('Boxplot of Market Capitalization (USD Billion)', fontsize=10)
plt.xlabel('Market Cap (USD Billion)', fontsize=12)
plt.show()
```



#3.6. Market Cap Quartile Distribution [3 Marks]

Display the quartile distribution of market capitalization using a violin plot.

```
# Market Cap distribution by quartile
plt.figure(figsize=(8,6))

sns.violinplot(mktg_df['Market Cap (USD Billion)'], color='lightgreen', inner='quartile')

plt.title('Market Capitalization Distribution by Quartile (USD Billion)', fontsize=10)
plt.xlabel('Market Cap (USD Billion)', fontsize=12)
plt.show()
```



#3.7. Cumulative Market Share Analysis [3 Marks]

Compute cumulative market share and visualize it with a line plot.

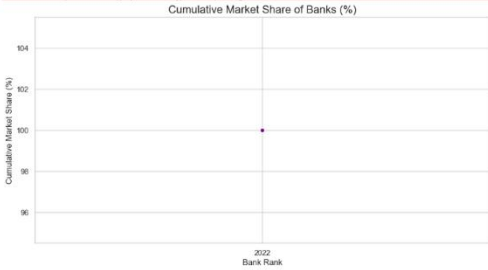
```
# Cumulative Market Share
mktg_df_sorted = mktg_df.sort_values(by='Market Cap (USD Billion)', ascending=False).reset_index(drop=True)
total_market_cap = mktg_df_sorted['Market Cap (USD Billion)'].cumsum()
mktg_df_sorted['Cumulative_Market_Share'] = total_market_cap / total_market_cap * 100

# Plot
plt.figure(figsize=(12,4))

sns.lineplot(mktg_df_sorted, x='Rank', y='Cumulative_Market_Share', markers='o', color='purple')

plt.title('Cumulative Market Share of Banks (%)', fontsize=10)
plt.xlabel('Rank', fontsize=12)
plt.ylabel('Cumulative Market Share (%)', fontsize=12)
plt.grid(True)
plt.show()
```

INFO:matplotlib.category:using categorical units to plot a list of strings that are all parsable as floats or dates. If these strings should be listed as numbers, cast to the appropriate data type before plotting.
INFO:matplotlib.category:using categorical units to plot a list of strings that are all parsable as floats or dates. If these strings should be listed as numbers, cast to the appropriate data type before plotting.



#3.8. Categorising Banks [3 Marks]

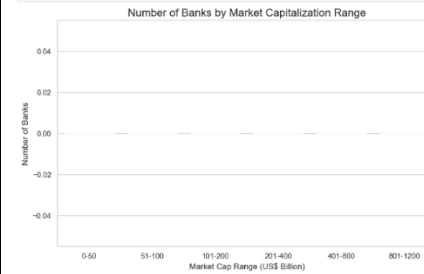
Categorize banks into market capitalization ranges and analyze their distribution using a bar chart.

```
# Market Cap Range Distribution
# Create market cap ranges
bins = [0, 50, 100, 200, 400, 800, 1200] # Adjust ranges based on dataset
labels = ['<50', '51-100', '101-200', '201-400', '401-800', '801-1200']
mktg_df['Market_Cap_Range'] = pd.cut(mktg_df['Market Cap (USD Billion)'], bins=bins, labels=labels, right=True)
range_counts = mktg_df['Market_Cap_Range'].value_counts().sort_index()

# Plot
plt.figure(figsize=(10,4))

sns.barplot(range_counts.index, range_counts.values, palette='magma')

plt.title('Number of Banks by Market Capitalization Range', fontsize=10)
plt.xlabel('Market Cap Range (USD Billion)', fontsize=12)
plt.ylabel('Number of Banks', fontsize=12)
plt.show()
```



#3.9. Visualize Market Share Distribution [3 Marks]

Calculate and display market share distribution among the top 10 banks using a pie chart.

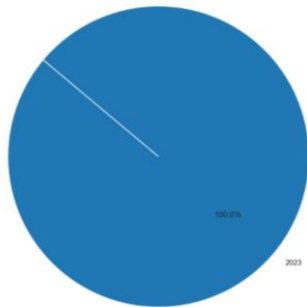
```
# Top 10 Banks Market Share Distribution
top10_banks = mktg_df.sort_values(by='Market Cap (USD Billion)', ascending=False).head(10)
top10_banks = top10_banks.dropna(subset=['Market Cap (USD Billion)'])
top10_banks['Market_Share_Percent'] = (top10_banks['Market Cap (USD Billion)'] / mktg_df['Market Cap (USD Billion)'].sum()) * 100

plt.figure(figsize=(10,10))

plt.pie(top10_banks['Market_Share_Percent'],
        labels=top10_banks['Bank Name'],
        autopct='%1.1f%%',
        startangle=90,
        colors=sns.color_palette('magma'))

plt.title('Market Share Distribution Among Top 10 Banks', fontsize=10)
plt.show()
```

Market Share Distribution Among Top 10 Banks



4. BANKING DATA ELT QUERYING

This ETL & querying phase computes advanced market-cap metrics — growth rates, consecutive-bank gaps, quartile and dominance scores — and classifies banks into market-share tiers and size segments. The results measure concentration, identify performance gaps, and feed visual summaries (charts and a dashboard) for actionable comparison and strategic insight.

```
#4. ETL and Querying (35 marks)

# Accounting your DataFrame to collect bank_df
import pandas as pd

# Sample DataFrame check
print(bank_df.head())

# 1. Top 5 Banks by Market Cap
top_banks = bank_df.sort_values(by='Market Cap (USD Billion)', ascending=False).head(5)
print("Top 5 Banks by Market Cap:")
print(top_banks[['Bank Name', 'Market Cap (USD Billion)']])

# 2. Count of banks in different market cap ranges
# Example ranges: (0, 10], (10, 50], (50, 100], (100, 500], (500, 1000]
bins = [10, 100, 500, 1000, 5000, 10000, 50000, 100000, 500000, 1000000]
labels = ["(0, 10]", "(10, 50]", "(50, 100]", "(100, 500]", "(500, 1000]", "(1000, 5000]", "(5000, 10000]", "(10000, 50000]", "(50000, 100000]", "(100000, 500000]", "(500000, 1000000]"]
bank_df['MarketCapRange'] = pd.cut(bank_df['Market Cap (USD Billion)'], bins=bins, labels=labels)

# 3. Range counts & bank_df['MarketCapRange'].value_counts().sort_index()
print("Count of banks by Market Cap Range:")
print(range_counts)

# 4. Average market cap
avg_market_cap = bank_df['Market Cap (USD Billion)'].mean()
print(f"Average Market Cap: {avg_market_cap}")

# 5. Banks with Above-Average Market Cap
above_avg_banks = bank_df[bank_df['Market Cap (USD Billion)'] > avg_market_cap]
print(f"Banks with Above-Average Market Cap:")
print(above_avg_banks[['Bank Name', 'Market Cap (USD Billion)']])

# 6. Banks with Below-Average Market Cap
below_avg_banks = bank_df[bank_df['Market Cap (USD Billion)'] < avg_market_cap]
print(f"Banks with Below-Average Market Cap:")
print(below_avg_banks[['Bank Name', 'Market Cap (USD Billion)']])
```

```
#4.2 Market Concentration Analysis (5 marks)

# Analyze Market Concentration and Categorize Banks Based on Market Share Tiers

# 1. Market Concentration Analysis
import numpy as np
import pandas as pd

# Sample DataFrame check
print(bank_df.head())

# 2. Categorize banks based on Market Share Tiers
def categorize_banks(bank_df):
    # Define market share tiers (e.g., Top 10%, Top 25%, Top 50%)
    tiers = ['Tier 1 (Top 10%)', 'Tier 2 (Top 25%)', 'Tier 3 (Top 50%)']
    # Assign tiers based on market share percentiles
    bank_df['MarketShareTier'] = bank_df['Market Share (%)'].apply(lambda x: tiers[0] if x <= 10 else tiers[1] if x <= 25 else tiers[2])
    return bank_df

# 3. Display the tier distribution
tier_counts = bank_df.groupby('MarketShareTier').value_counts()
print("Market Share Tier Distribution:")
print(tier_counts)

# 4. Visualize the tier distribution
plt.figure(figsize=(10, 5))
plt.bar(tier_counts.index, tier_counts.values, color='lightblue')
plt.xlabel('Market Share Tier')
plt.ylabel('Number of Banks')
plt.title('Market Share Tier Distribution')
plt.show()
```



```
#4.1 Market Capitalization Analysis (3 marks)

# Perform Advanced Market Capitalization Analysis with Growth Metrics

# 1. Market Capitalization Analysis
import pandas as pd

# Sample DataFrame check
print(bank_df.head())

# 2. Market Capitalization Metrics
# Ensure Market Cap is numeric
bank_df['Market Cap (USD Billion)'] = pd.to_numeric(bank_df['Market Cap (USD Billion)'], errors='coerce')

# Drop rows with missing Market Cap
bank_df = bank_df.dropna(subset=['Market Cap (USD Billion)'])

# Rank banks by Market Cap
bank_df['Rank'] = bank_df['Market Cap (USD Billion)'].rank(method='dense', ascending=False).astype(int)

# Compute market share
total_market_cap = bank_df['Market Cap (USD Billion)'].sum()
bank_df['MarketSharePercent'] = (bank_df['Market Cap (USD Billion)'] / total_market_cap) * 100
bank_df['CumulativeMarketShare'] = bank_df['MarketSharePercent'].cumsum()

# Compute Market Cap Gap
bank_df['MarketCapGap'] = bank_df['Market Cap (USD Billion)'].diff().fillna(0)

# Compute Market Cap Growth Rate
bank_df['MarketCapGrowthRate'] = bank_df['MarketCapGap'] / bank_df['MarketCapGap'].abs().fillna(1)

# Compute Market Cap Quartiles
bank_df['MarketCapQuartile'] = pd.qcut(bank_df['Market Cap (USD Billion)'], 4, labels=['Q1', 'Q2', 'Q3', 'Q4'])

# 3. Market Capitalization Distribution
# Compute Market Cap Quartiles
bank_df['MarketCapQuartile'] = pd.qcut(bank_df['Market Cap (USD Billion)'], 4, labels=['Q1', 'Q2', 'Q3', 'Q4'])

# 4. Market Capitalization Distribution
# Compute Market Cap Quartiles
bank_df['MarketCapQuartile'] = pd.qcut(bank_df['Market Cap (USD Billion)'], 4, labels=['Q1', 'Q2', 'Q3', 'Q4'])
```

```
#4.3 Market Capitalization Distribution (3 marks)

# Examine Statistical Distribution of Market Capitalization Using Quartile Analysis

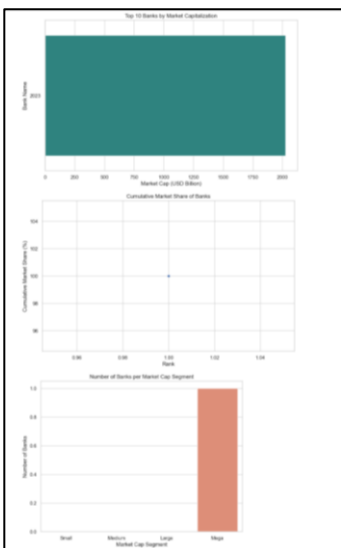
# 1. Market Capitalization Distribution
import pandas as pd

# Sample DataFrame check
print(bank_df.head())

# 2. Market Capitalization Distribution
# Compute Market Cap Quartiles
bank_df['MarketCapQuartile'] = pd.qcut(bank_df['Market Cap (USD Billion)'], 4, labels=['Q1', 'Q2', 'Q3', 'Q4'])

# 3. Market Capitalization Distribution
# Compute Market Cap Quartiles
bank_df['MarketCapQuartile'] = pd.qcut(bank_df['Market Cap (USD Billion)'], 4, labels=['Q1', 'Q2', 'Q3', 'Q4'])

# 4. Market Capitalization Distribution
# Compute Market Cap Quartiles
bank_df['MarketCapQuartile'] = pd.qcut(bank_df['Market Cap (USD Billion)'], 4, labels=['Q1', 'Q2', 'Q3', 'Q4'])
```



5. CONCLUSIONS

5.1. Recommendations for Tracking & Comparing Market Capitalisation

- Regular monitoring of market capitalization helps identify shifts in global banking dominance, especially for top institutions.
- Benchmarking top banks helps evaluate:
 - Competitive position
 - Growth rate changes
 - Market stability
- Including historical data (year-over-year growth) will allow firms to forecast industry trends and identify rising competitors among global banks.

5.2. Cross-Currency Analysis for Better Benchmarking

- Since banks operate globally, evaluating market cap in multiple currencies (USD, GBP, EUR, INR) provides **regionally consistent comparisons**.
- Currency fluctuations impact valuations; hence multi-currency tracking:
 - Neutralises conversion distortions
 - Supports global investment decisions
 - Improves comparability for multinational institutions

5.3. Continuous Monitoring of Market Share Concentration

- The market is highly concentrated among top banks based on your results.
- Continuous monitoring helps identify:
 - Opportunities for mid-tier banks to expand
 - Early signs of competition weakening or strengthening
 - Regulatory concerns if dominance becomes excessive
- Segment-wise tracking (Small, Medium, Large, Mega) can reveal upward movement of emerging banks.

5.4. Identifying Expansion Opportunities Using Tier Gaps & Regional Trends

- The gap between bank tiers (Small vs Medium vs Large vs Mega) highlights regions where growth potential exists.
- Regions with fewer large banks may offer expansion opportunities for:
 - Regional banks aiming to become national players
 - Large banks planning cross-border entry
- Market size gaps can indicate:
 - Underdeveloped banking markets
 - Emerging economies with increasing demand
 - High-growth zones for digital banking expansion